

Alzheimer's Multiclass Classification – Inception

```
# This Python 3 environment comes with many helpful analytics  
libraries installed  
# It is defined by the kaggle/python Docker image:  
https://github.com/kaggle/docker-python  
# For example, here's several helpful packages to load  
  
import numpy as np # linear algebra  
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)  
  
# Input data files are available in the read-only "../input/"  
directory  
# For example, running this (by clicking run or pressing Shift+Enter)  
will list all files under the input directory  
  
import os  
for dirname, _, filenames in os.walk('/kaggle/input'):  
    for filename in filenames:  
        print(os.path.join(dirname, filename))  
  
# You can write up to 20GB to the current directory (/kaggle/working/)  
that gets preserved as output when you create a version using "Save &  
Run All"  
# You can also write temporary files to /kaggle/temp/, but they won't  
be saved outside of the current session
```

Data Preprocessing

Import libraries & Load grayscale data

```
import os  
import cv2  
import numpy as np  
from collections import Counter  
import matplotlib.pyplot as plt  
  
base_dir = '/kaggle/input/alzheimers-multiclass-dataset-equal-and-  
augmented/combined_images'  
categories = ['VeryMildDemented', 'NonDemented', 'ModerateDemented',  
'MildDemented']  
  
# Load grayscale images  
images = []  
labels = []  
  
for category in categories:  
    path = os.path.join(base_dir, category)
```

```

for img_name in os.listdir(path):
    img_path = os.path.join(path, img_name)
    try:
        img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (128, 128))
        images.append(img)
        labels.append(categories.index(category))
    except Exception as e:
        print(f"Error: {e}")

images = np.array(images)
labels = np.array(labels)
print(f"Loaded shape: {images.shape}")

# Add channel dimension & normalize
images = np.expand_dims(images, axis=-1)
images = images.astype('float32') / 255.0
print(f"Post-norm shape: {images.shape}, range: [{images.min():.3f}, {images.max():.3f}]")

Loaded shape: (44000, 128, 128)
Post-norm shape: (44000, 128, 128, 1), range: [0.000, 1.000]

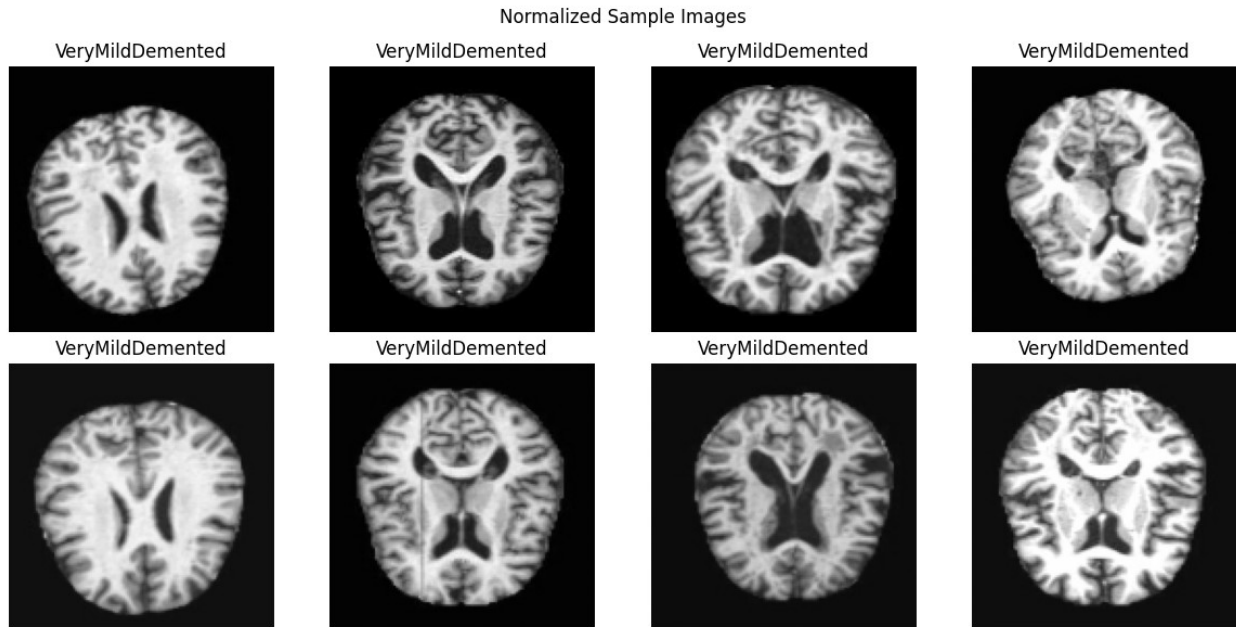
```

Visualize normalized samples :

```

fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for i in range(4):
    axes[0, i].imshow(images[i].squeeze(), cmap='gray')
    axes[0, i].set_title(f'{categories[labels[i]]}')
    axes[0, i].axis('off')
    axes[1, i].imshow(images[i+4].squeeze(), cmap='gray', vmin=0,
vmax=1)
    axes[1, i].set_title(f'{categories[labels[i+4]]}')
    axes[1, i].axis('off')
plt.suptitle('Normalized Sample Images')
plt.tight_layout()
plt.show()

```



Train-test split with numpy one-hot encoding

```
from sklearn.model_selection import train_test_split
import numpy as np

# One-hot encoding with numpy (avoids TF/protobuf errors)
y = np.eye(4)[labels]

X_train, X_test, y_train, y_test = train_test_split(
    images, y, test_size=0.2, stratify=labels, random_state=42
)

print(f"Train: {X_train.shape[0]}, Test: {X_test.shape[0]}")
print(f"y_train shape: {y_train.shape}")
print(f"Class distribution - Train: {np.sum(y_train, axis=0)}")
print(f"Class distribution - Test: {np.sum(y_test, axis=0)}")

Train: 35200, Test: 8800
y_train shape: (35200, 4)
Class distribution - Train: [ 8960. 10240.  8000.  8000.]
Class distribution - Test: [2240. 2560. 2000. 2000.]
```

Model Building & Training - InceptionV3

GPU & Mixed Precision Setup :

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
import tensorflow as tf
```

```

print("TensorFlow:", tf.__version__)
print("CUDA Built:", tf.test.is_built_with_cuda())
print("GPUs Available:", tf.config.list_physical_devices('GPU'))

# Enable mixed precision for faster training (same as VGG)
from tensorflow.keras.mixed_precision import set_global_policy
set_global_policy('mixed_float16')

# GPU memory growth
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
    tf.config.set_visible_devices(gpus[0], 'GPU')
    print("Logical GPUs:",
          tf.config.experimental.list_logical_devices('GPU'))

# GPU test
with tf.device('/GPU:0'):
    @tf.function
    def test():
        return tf.matmul(tf.random.uniform((1000, 1000)),
                           tf.random.uniform((1000, 1000)))
    print("GPU test successful:", test().shape)

TensorFlow: 2.19.0
CUDA Built: True
GPUs Available: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1',
device_type='GPU')]
Logical GPUs: [LogicalDevice(name='/device:GPU:0', device_type='GPU')]
GPU test successful: (1000, 1000)

```

Disk Generators for InceptionV3

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.inception_v3 import
preprocess_input

base_dir = '/kaggle/input/alzheimers-multiclass-dataset-equal-and-
augmented/combined_images'

# InceptionV3 uses 299x299 input size
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    base_dir,

```

```

        target_size=(299, 299), # InceptionV3 standard input
        color_mode='rgb',
        batch_size=64,
        class_mode='categorical',
        subset='training',
        shuffle=True
    )

    val_generator = train_datagen.flow_from_directory(
        base_dir,
        target_size=(299, 299),
        color_mode='rgb',
        batch_size=64,
        class_mode='categorical',
        subset='validation'
    )

    test_datagen =
    ImageDataGenerator(preprocessing_function=preprocess_input)
    test_generator = test_datagen.flow_from_directory(
        base_dir,
        target_size=(299, 299),
        color_mode='rgb',
        batch_size=64,
        class_mode='categorical',
        shuffle=False
    )

    print(f"Train batches: {train_generator.samples // 64}")
    print("Classes:", train_generator.class_indices)

    Found 35200 images belonging to 4 classes.
    Found 8800 images belonging to 4 classes.
    Found 44000 images belonging to 4 classes.
    Train batches: 550
    Classes: {'MildDemented': 0, 'ModerateDemented': 1, 'NonDemented': 2,
    'VeryMildDemented': 3}

```

Build InceptionV3 Model:

```

from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense,
Dropout
from tensorflow.keras.optimizers import Adam

# Load InceptionV3 base (frozen weights)
base_model = InceptionV3(
    weights='imagenet',
    include_top=False,

```

```

    input_shape=(299, 299, 3)
)
base_model.trainable = False

# Build model
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(512, activation='relu'),
    Dropout(0.5), # Same dropout rate
    Dense(4, activation='softmax', dtype='float32') # 4 classes
])

# Compile with same optimizer & learning rate
model.compile(
    optimizer=Adam(learning_rate=0.0001), # Same LR as VGG
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_v3/inception_v3_weights_tf_dim_ordering_tf_kernels_notop.h5
 87910968/87910968 _____ 0s 0us/step

Model: "sequential"

Layer (type) Param #	Output Shape	
inception_v3 (Functional) 21,802,784	(None, 8, 8, 2048)	
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	
dense (Dense) 1,049,088	(None, 512)	
dropout (Dropout)	(None, 512)	

0				
		dense_1 (Dense)	(None, 4)	
2,052				
Total params: 22,853,924 (87.18 MB)				
Trainable params: 1,051,140 (4.01 MB)				
Non-trainable params: 21,802,784 (83.17 MB)				

Callbacks

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
ReduceLROnPlateau

callbacks = [
    EarlyStopping(
        patience=10,
        restore_best_weights=True,
        monitor='val_accuracy'
    ),
    ModelCheckpoint(
        'best_inception_alzheimer.h5',
        save_best_only=True,
        monitor='val_accuracy'
    ),
    ReduceLROnPlateau(
        factor=0.5,
        patience=5,
        min_lr=1e-7,
        monitor='val_loss'
    )
]
```

Train InceptionV3 :

```
history = model.fit(
    train_generator,
    epochs=50,
    validation_data=val_generator,
    validation_freq=5,
    callbacks=callbacks,
    verbose=1
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/
data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

Epoch 1/50

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

```
I0000 00:00:1767434677.055377      135 service.cc:152] XLA service
0x792a840589d0 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
```

```
I0000 00:00:1767434677.055422      135 service.cc:160] StreamExecutor
device (0): Tesla T4, Compute Capability 7.5
```

```
I0000 00:00:1767434679.501122      135 cuda_dnn.cc:529] Loaded cuDNN
version 91002
```

```
1/550 _____ 3:36:51 24s/step - accuracy: 0.2344 -
loss: 1.7647
```

```
I0000 00:00:1767434692.686664      135 device_compiler.h:188] Compiled
cluster using XLA! This line is logged at most once for the lifetime
of the process.
```

```
550/550 _____ 0s 214ms/step - accuracy: 0.4836 - loss:
1.1411
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/callbacks/
early_stopping.py:153: UserWarning: Early stopping conditioned on
metric `val_accuracy` which is not available. Available metrics are:
accuracy, loss
```

```
current = self.get_monitor_value(logs)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/callbacks/model_chec
kpoint.py:302: UserWarning: Can save best model only with val_accuracy
available.
```

```
if self._should_save_model(epoch, batch, logs, filepath):
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 142s 215ms/step - accuracy: 0.4838 -
loss: 1.1409 - learning_rate: 1.0000e-04
Epoch 2/50
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/callbacks/
callback_list.py:145: UserWarning: Learning rate reduction is
```


conditioned on metric `val\_loss` which is not available. Available metrics are: accuracy, loss, learning_rate.
callback.on_epoch_end(epoch, logs)

550/550 _____ 0s 211ms/step - accuracy: 0.6332 - loss: 0.8278

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 117s 212ms/step - accuracy: 0.6332 - loss: 0.8278 - learning_rate: 1.0000e-04
Epoch 3/50

550/550 _____ 0s 213ms/step - accuracy: 0.6706 - loss: 0.7510

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 118s 214ms/step - accuracy: 0.6706 - loss: 0.7509 - learning_rate: 1.0000e-04
Epoch 4/50

550/550 _____ 0s 216ms/step - accuracy: 0.6877 - loss: 0.7053

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 119s 217ms/step - accuracy: 0.6877 - loss: 0.7053 - learning_rate: 1.0000e-04
Epoch 5/50

550/550 _____ 0s 215ms/step - accuracy: 0.7098 - loss: 0.6639

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 168s 305ms/step - accuracy: 0.7098 - loss: 0.6639 - val_accuracy: 0.7387 - val_loss: 0.6341 -

```
learning_rate: 1.0000e-04
Epoch 6/50
550/550 _____ 0s 215ms/step - accuracy: 0.7247 - loss:
0.6323
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 119s 216ms/step - accuracy: 0.7247 -
loss: 0.6323 - learning_rate: 1.0000e-04
Epoch 7/50
550/550 _____ 0s 208ms/step - accuracy: 0.7397 - loss:
0.6058
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 115s 209ms/step - accuracy: 0.7397 -
loss: 0.6058 - learning_rate: 1.0000e-04
Epoch 8/50
550/550 _____ 0s 212ms/step - accuracy: 0.7485 - loss:
0.5876
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 117s 213ms/step - accuracy: 0.7485 -
loss: 0.5876 - learning_rate: 1.0000e-04
Epoch 9/50
550/550 _____ 0s 207ms/step - accuracy: 0.7598 - loss:
0.5639
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 114s 208ms/step - accuracy: 0.7598 -
loss: 0.5639 - learning_rate: 1.0000e-04
Epoch 10/50
```

550/550 ————— 0s 195ms/step - accuracy: 0.7692 - loss: 0.5444

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 143s 260ms/step - accuracy: 0.7692 - loss: 0.5444 - val_accuracy: 0.7718 - val_loss: 0.5443 - learning_rate: 1.0000e-04

Epoch 11/50

550/550 ————— 0s 207ms/step - accuracy: 0.7764 - loss: 0.5325

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 114s 208ms/step - accuracy: 0.7765 - loss: 0.5325 - learning_rate: 1.0000e-04

Epoch 12/50

550/550 ————— 0s 207ms/step - accuracy: 0.7893 - loss: 0.5100

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 115s 208ms/step - accuracy: 0.7893 - loss: 0.5100 - learning_rate: 1.0000e-04

Epoch 13/50

550/550 ————— 0s 220ms/step - accuracy: 0.7949 - loss: 0.4949

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 122s 221ms/step - accuracy: 0.7949 - loss: 0.4949 - learning_rate: 1.0000e-04

Epoch 14/50

550/550 ————— 0s 223ms/step - accuracy: 0.8020 - loss: 0.4797

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 123s 223ms/step - accuracy: 0.8020 -
loss: 0.4797 - learning_rate: 1.0000e-04

Epoch 15/50

550/550 _____ 0s 213ms/step - accuracy: 0.8119 - loss:
0.4657

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 152s 276ms/step - accuracy: 0.8119 -
loss: 0.4657 - val_accuracy: 0.8145 - val_loss: 0.4622 -
learning_rate: 1.0000e-04

Epoch 16/50

550/550 _____ 0s 213ms/step - accuracy: 0.8136 - loss:
0.4514

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 118s 214ms/step - accuracy: 0.8136 -
loss: 0.4514 - learning_rate: 1.0000e-04

Epoch 17/50

550/550 _____ 0s 212ms/step - accuracy: 0.8219 - loss:
0.4362

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 117s 213ms/step - accuracy: 0.8219 -
loss: 0.4362 - learning_rate: 1.0000e-04

Epoch 18/50

550/550 _____ 0s 208ms/step - accuracy: 0.8271 - loss:
0.4214

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format

is considered legacy. We recommend using instead the native Keras format, e.g. ``model.save('my_model.keras')`` or ``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 115s 209ms/step - accuracy: 0.8271 - loss: 0.4214 - learning_rate: 1.0000e-04

Epoch 19/50

550/550 _____ 0s 208ms/step - accuracy: 0.8326 - loss: 0.4128

WARNING:absl:You are saving your model as an HDF5 file via ``model.save()`` or ``keras.saving.save_model(model)``. This file format is considered legacy. We recommend using instead the native Keras format, e.g. ``model.save('my_model.keras')`` or ``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 115s 208ms/step - accuracy: 0.8326 - loss: 0.4128 - learning_rate: 1.0000e-04

Epoch 20/50

550/550 _____ 0s 205ms/step - accuracy: 0.8409 - loss: 0.3991

WARNING:absl:You are saving your model as an HDF5 file via ``model.save()`` or ``keras.saving.save_model(model)``. This file format is considered legacy. We recommend using instead the native Keras format, e.g. ``model.save('my_model.keras')`` or ``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 150s 273ms/step - accuracy: 0.8409 - loss: 0.3991 - val_accuracy: 0.8291 - val_loss: 0.4258 - learning_rate: 1.0000e-04

Epoch 21/50

550/550 _____ 0s 188ms/step - accuracy: 0.8425 - loss: 0.3930

WARNING:absl:You are saving your model as an HDF5 file via ``model.save()`` or ``keras.saving.save_model(model)``. This file format is considered legacy. We recommend using instead the native Keras format, e.g. ``model.save('my_model.keras')`` or ``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 104s 189ms/step - accuracy: 0.8425 - loss: 0.3930 - learning_rate: 1.0000e-04

Epoch 22/50

550/550 _____ 0s 207ms/step - accuracy: 0.8516 - loss: 0.3761

WARNING:absl:You are saving your model as an HDF5 file via ``model.save()`` or ``keras.saving.save_model(model)``. This file format is considered legacy. We recommend using instead the native Keras

```
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 114s 208ms/step - accuracy: 0.8516 -  
loss: 0.3761 - learning_rate: 1.0000e-04
```

```
Epoch 23/50
```

```
550/550 _____ 0s 204ms/step - accuracy: 0.8552 - loss:  
0.3680
```

```
WARNING:absl:You are saving your model as an HDF5 file via  
`model.save()` or `keras.saving.save_model(model)`. This file format  
is considered legacy. We recommend using instead the native Keras  
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 113s 205ms/step - accuracy: 0.8552 -  
loss: 0.3680 - learning_rate: 1.0000e-04
```

```
Epoch 24/50
```

```
550/550 _____ 0s 205ms/step - accuracy: 0.8571 - loss:  
0.3625
```

```
WARNING:absl:You are saving your model as an HDF5 file via  
`model.save()` or `keras.saving.save_model(model)`. This file format  
is considered legacy. We recommend using instead the native Keras  
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 113s 206ms/step - accuracy: 0.8571 -  
loss: 0.3625 - learning_rate: 1.0000e-04
```

```
Epoch 25/50
```

```
550/550 _____ 0s 203ms/step - accuracy: 0.8647 - loss:  
0.3480
```

```
WARNING:absl:You are saving your model as an HDF5 file via  
`model.save()` or `keras.saving.save_model(model)`. This file format  
is considered legacy. We recommend using instead the native Keras  
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 148s 268ms/step - accuracy: 0.8647 -  
loss: 0.3480 - val_accuracy: 0.8517 - val_loss: 0.3797 -  
learning_rate: 1.0000e-04
```

```
Epoch 26/50
```

```
550/550 _____ 0s 204ms/step - accuracy: 0.8692 - loss:  
0.3390
```

```
WARNING:absl:You are saving your model as an HDF5 file via  
`model.save()` or `keras.saving.save_model(model)`. This file format  
is considered legacy. We recommend using instead the native Keras  
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 113s 205ms/step - accuracy: 0.8692 -  
loss: 0.3390 - learning_rate: 1.0000e-04  
Epoch 27/50  
550/550 _____ 0s 196ms/step - accuracy: 0.8745 - loss:  
0.3272
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 109s 197ms/step - accuracy: 0.8745 -  
loss: 0.3272 - learning_rate: 1.0000e-04  
Epoch 28/50  
550/550 _____ 0s 227ms/step - accuracy: 0.8764 - loss:  
0.3221
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 125s 228ms/step - accuracy: 0.8764 -  
loss: 0.3221 - learning_rate: 1.0000e-04  
Epoch 29/50  
550/550 _____ 0s 205ms/step - accuracy: 0.8770 - loss:  
0.3154
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 113s 206ms/step - accuracy: 0.8770 -  
loss: 0.3154 - learning_rate: 1.0000e-04  
Epoch 30/50  
550/550 _____ 0s 206ms/step - accuracy: 0.8839 - loss:  
0.3023
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 142s 258ms/step - accuracy: 0.8839 -  
loss: 0.3023 - val_accuracy: 0.8639 - val_loss: 0.3438 -  
learning_rate: 1.0000e-04
```

Epoch 31/50
550/550 ————— 0s 207ms/step - accuracy: 0.8873 - loss: 0.2929

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 114s 207ms/step - accuracy: 0.8873 - loss: 0.2930 - learning_rate: 1.0000e-04

Epoch 32/50
550/550 ————— 0s 211ms/step - accuracy: 0.8880 - loss: 0.2907

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 117s 212ms/step - accuracy: 0.8880 - loss: 0.2907 - learning_rate: 1.0000e-04

Epoch 33/50
550/550 ————— 0s 212ms/step - accuracy: 0.8901 - loss: 0.2874

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 117s 213ms/step - accuracy: 0.8901 - loss: 0.2874 - learning_rate: 1.0000e-04

Epoch 34/50
550/550 ————— 0s 211ms/step - accuracy: 0.8954 - loss: 0.2732

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 ————— 117s 212ms/step - accuracy: 0.8954 - loss: 0.2732 - learning_rate: 1.0000e-04

Epoch 35/50
550/550 ————— 0s 228ms/step - accuracy: 0.8995 - loss: 0.2646

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 162s 294ms/step - accuracy: 0.8995 -
loss: 0.2646 - val_accuracy: 0.8643 - val_loss: 0.3352 -
learning_rate: 1.0000e-04
Epoch 36/50

550/550 _____ 0s 245ms/step - accuracy: 0.9000 - loss:
0.2631

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 135s 246ms/step - accuracy: 0.9000 -
loss: 0.2630 - learning_rate: 1.0000e-04
Epoch 37/50

550/550 _____ 0s 248ms/step - accuracy: 0.9029 - loss:
0.2545

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 137s 249ms/step - accuracy: 0.9029 -
loss: 0.2545 - learning_rate: 1.0000e-04
Epoch 38/50

550/550 _____ 0s 232ms/step - accuracy: 0.9090 - loss:
0.2495

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

550/550 _____ 128s 233ms/step - accuracy: 0.9090 -
loss: 0.2495 - learning_rate: 1.0000e-04
Epoch 39/50

550/550 _____ 0s 227ms/step - accuracy: 0.9152 - loss:
0.2305

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format

is considered legacy. We recommend using instead the native Keras format, e.g. ``model.save('my_model.keras')`` or ``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 125s 228ms/step - accuracy: 0.9152 -
loss: 0.2306 - learning_rate: 1.0000e-04

Epoch 40/50

550/550 _____ 0s 246ms/step - accuracy: 0.9147 - loss:
0.2332

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 181s 329ms/step - accuracy: 0.9147 -
loss: 0.2332 - val_accuracy: 0.8848 - val_loss: 0.2974 -
learning_rate: 1.0000e-04

Epoch 41/50

550/550 _____ 0s 230ms/step - accuracy: 0.9156 - loss:
0.2312

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 127s 231ms/step - accuracy: 0.9156 -
loss: 0.2312 - learning_rate: 1.0000e-04

Epoch 42/50

550/550 _____ 0s 233ms/step - accuracy: 0.9177 - loss:
0.2260

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 129s 234ms/step - accuracy: 0.9177 -
loss: 0.2260 - learning_rate: 1.0000e-04

Epoch 43/50

550/550 _____ 0s 223ms/step - accuracy: 0.9160 - loss:
0.2249

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras

format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 123s 224ms/step - accuracy: 0.9160 -
loss: 0.2249 - learning_rate: 1.0000e-04
Epoch 44/50

550/550 _____ 0s 218ms/step - accuracy: 0.9197 - loss:
0.2165

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 121s 219ms/step - accuracy: 0.9197 -
loss: 0.2165 - learning_rate: 1.0000e-04
Epoch 45/50

550/550 _____ 151s 274ms/step - accuracy: 0.9247 -
loss: 0.2102 - val_accuracy: 0.8802 - val_loss: 0.3035 -
learning_rate: 1.0000e-04

Epoch 46/50
550/550 _____ 0s 231ms/step - accuracy: 0.9253 - loss:
0.2059

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 128s 232ms/step - accuracy: 0.9253 -
loss: 0.2059 - learning_rate: 1.0000e-04
Epoch 47/50

550/550 _____ 0s 220ms/step - accuracy: 0.9248 - loss:
0.2021

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. ``model.save('my_model.keras')`` or
``keras.saving.save_model(model, 'my_model.keras')``.

550/550 _____ 122s 221ms/step - accuracy: 0.9248 -
loss: 0.2021 - learning_rate: 1.0000e-04
Epoch 48/50

550/550 _____ 0s 225ms/step - accuracy: 0.9276 - loss:
0.1970

WARNING:absl:You are saving your model as an HDF5 file via
``model.save()`` or ``keras.saving.save_model(model)``. This file format

is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

550/550 _____ 125s 226ms/step - accuracy: 0.9276 - loss: 0.1970 - learning_rate: 1.0000e-04

Epoch 49/50

550/550 _____ 0s 220ms/step - accuracy: 0.9293 - loss: 0.1926

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

550/550 _____ 121s 221ms/step - accuracy: 0.9293 - loss: 0.1926 - learning_rate: 1.0000e-04

Epoch 50/50

550/550 _____ 0s 220ms/step - accuracy: 0.9278 - loss: 0.1937

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

550/550 _____ 160s 290ms/step - accuracy: 0.9278 - loss: 0.1937 - val_accuracy: 0.8980 - val_loss: 0.2634 - learning_rate: 1.0000e-04

Evaluation

Plot Training Curves :

```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc', linewidth=2)
plt.plot(history.history['val_accuracy'], label='Val Acc',
linewidth=2)
plt.title('InceptionV3 Accuracy', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

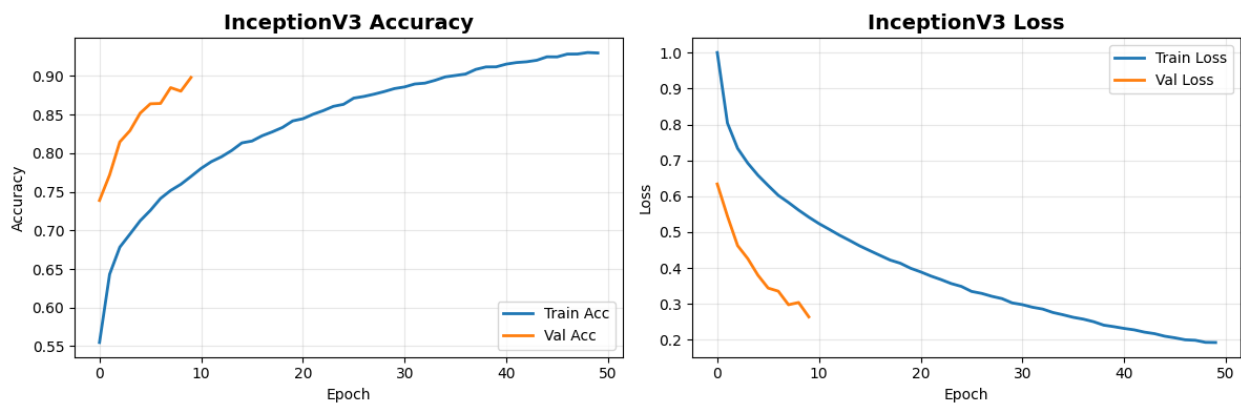
```

plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', linewidth=2)
plt.plot(history.history['val_loss'], label='Val Loss', linewidth=2)
plt.title('InceptionV3 Loss', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



Test Set Evaluation :

```

test_generator.reset()
test_loss, test_acc = model.evaluate(test_generator, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")

# Predictions
y_pred_prob = model.predict(test_generator)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = test_generator.classes

print("\n" + "="*60)
print("CLASSIFICATION REPORT - InceptionV3")
print("="*60)
print(classification_report(
    y_true,
    y_pred,
    target_names=list(train_generator.class_indices.keys()),
    digits=4
))

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/
data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.

```

```

self._warn_if_super_not_called()

```

Test Accuracy: 0.9540

Test Loss: 0.1546

688/688 ————— 163s 226ms/step

===== CLASSIFICATION REPORT - InceptionV3 =====

	precision	recall	f1-score	support
MildDemented	0.9690	0.9617	0.9653	10000
ModerateDemented	0.9982	0.9992	0.9987	10000
NonDemented	0.9600	0.9155	0.9373	12800
VeryMildDemented	0.8980	0.9507	0.9236	11200
accuracy			0.9540	44000
macro avg	0.9563	0.9568	0.9562	44000
weighted avg	0.9549	0.9540	0.9541	44000

Confusion Matrix :

```

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=list(train_generator.class_indices.keys()),
    yticklabels=list(train_generator.class_indices.keys()),
    cbar_kws={'label': 'Count'}
)
plt.title('Confusion Matrix - InceptionV3', fontsize=16,
fontweight='bold')
plt.ylabel('True Label', fontsize=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.tight_layout()
plt.show()

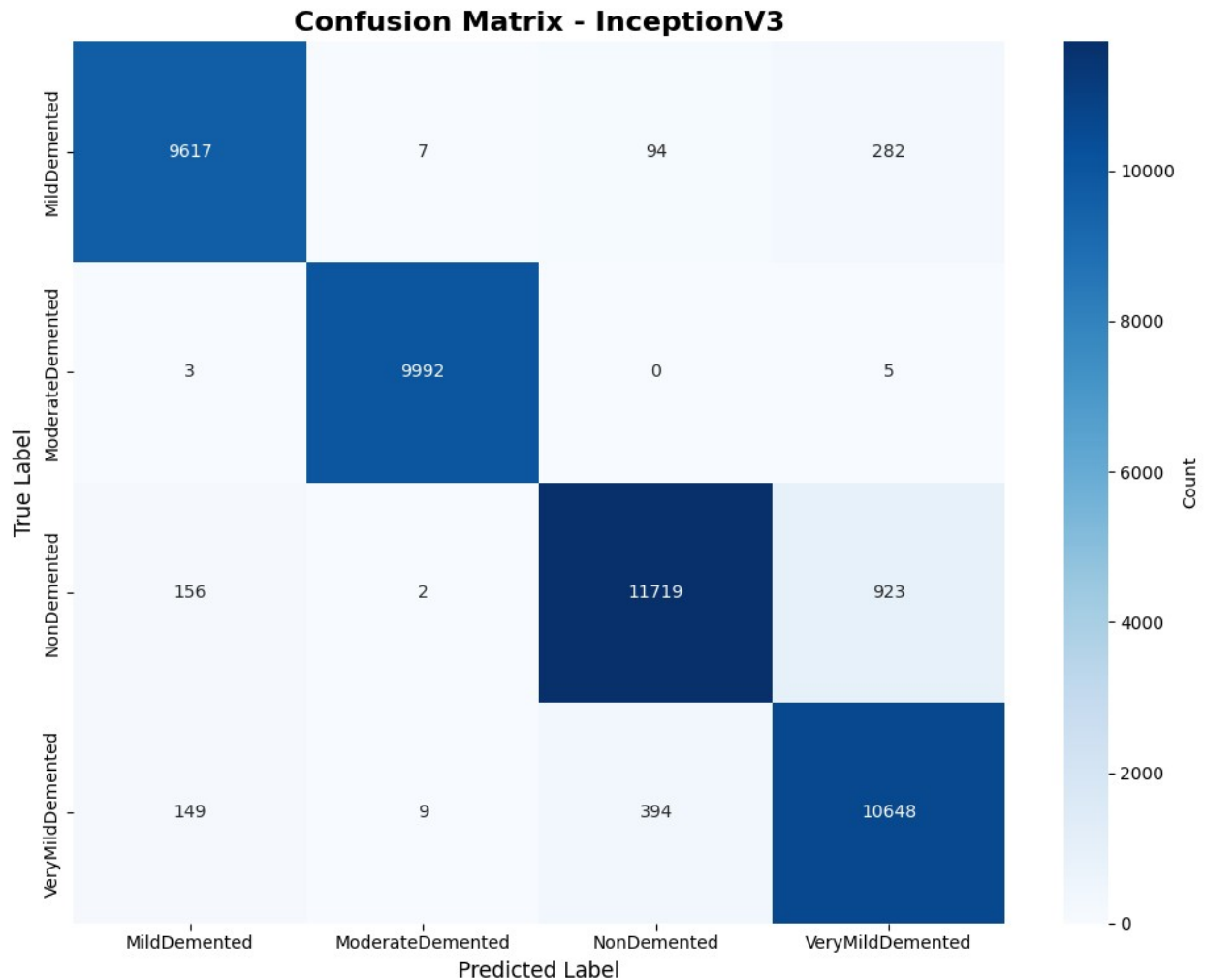
# Per-class accuracy
print("\n" + "="*60)

```

```

print("PER-CLASS ACCURACY")
print("="*60)
for i, class_name in enumerate(train_generator.class_indices.keys()):
    class_acc = cm[i, i] / cm[i].sum()
    print(f"{class_name:20s}: {class_acc:.4f} ({cm[i, i]}/{cm[i].sum()})")

```



===== PER-CLASS ACCURACY =====

```

MildDemented      : 0.9617 (9617/10000)
ModerateDemented  : 0.9992 (9992/10000)
NonDemented       : 0.9155 (11719/12800)
VeryMildDemented  : 0.9507 (10648/11200)

```

Model Saving :

```
# Save as native Keras (.keras)
model.save('alzheimer_inceptionv3.keras')
print("✓ Saved: alzheimer_inceptionv3.keras")

# Save as legacy HDF5 (.h5)
model.save('alzheimer_inceptionv3.h5')
print("✓ Saved: alzheimer_inceptionv3.h5")

# Verify saves
print("\nModel Files:")
import os
for f in ['alzheimer_inceptionv3.keras', 'alzheimer_inceptionv3.h5',
'best_inception_alzheimer.h5']:
    if os.path.exists(f):
        size = os.path.getsize(f) / (1024**2)
        print(f" - {f}: {size:.1f} MB")

# Cell 4.2: Create ZIP for Download
import zipfile

models = [
    'alzheimer_inceptionv3.h5',
    'alzheimer_inceptionv3.keras',
    'best_inception_alzheimer.h5'
]

zip_path = '/kaggle/working/alzheimer_inceptionv3_models.zip'
with zipfile.ZipFile(zip_path, 'w') as zf:
    for m in models:
        if os.path.exists(m):
            zf.write(m, m)

print(f"\n✓ Zipped: {zip_path}")
print("→ Download from Kaggle Output panel!")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
✓ Saved: alzheimer_inceptionv3.keras
✓ Saved: alzheimer_inceptionv3.h5
```

Model Files:

- alzheimer_inceptionv3.keras: 104.3 MB
- alzheimer_inceptionv3.h5: 95.8 MB
- best_inception_alzheimer.h5: 95.8 MB

✓ Zipped: /kaggle/working/alzheimer_inceptionv3_models.zip
→ Download from Kaggle Output panel!